

KTH Royal Institute of Technology

Software Engineering Fundamentals (DD2440)

Report for Assignment 4

Group 19:

Vadim El Guedj
Elias Gaghlasian
Elias Hollstrand
Fabian Holm
Jonathan Skantz

March 3, 2026

Contents

1 Project	1
2 Onboarding Experience	1
3 Effort Spent	2
4 Overview of Issues and Work Done	3
4.1 First Issue	3
4.2 Second Issue	3
5 Requirements for the New Features	4
6 Code changes	5
7 Architecture of the Program	6
8 UML	7
8.1 UML Class Diagram for Issue #572	7
8.2 UML Class Diagram for Issue #929	8
9 Overall Experience	9
9.1 Our Work in Context with Best Software Engineering Practices	9

1 Project

Name: TagStudio

URL: <https://github.com/TagStudioDev/TagStudio>

From the readme:

“TagStudio is a photo & file organization application with an underlying tag-based system that focuses on giving freedom and flexibility to the user. No proprietary programs or formats, no sea of sidecar files, and no complete upheaval of your filesystem structure.”

2 Onboarding Experience

For this assignment, we chose to work with a different open-source project than in Assignment 3. We decided to switch partly because the new project felt easier to understand architecturally, but also because the available issues seemed better scoped and more suitable for the goals of this assignment.

Compared to in Assignment 3, the onboarding process was largely similar here. The project provides clear setup instructions and contribution guidelines. However, we feel that this project has better documentation than the one we used in Assignment 3. The setup instructions were straightforward to follow, and we did not encounter any major technical issues during installation or initial configuration. Furthermore, the contributing guidelines were clear and explicit about workflow, coding standards, and other expectations which made it easy to understand how to go about contributing.

3 Effort Spent

	Elias H	Elias G	Fabian	Vadim	Jonathan
1. Plenary discussions/meetings: - Initial meeting - Defining and splitting up work - Discussing complexity of the first issue - Picking a second issue	3 h 15 min	2 h	2 h	2 h	3 h 15 min
2. Discussions within parts of the group: - Defining requirements, scope, and implementation of the first issue - Discussing implementation pros and cons	1 h 30 min	2 h	2 h	2 h	3 h
3. Reading docs: - Contributing - Developing - Style guide - Research how testing works in Python - Research relevant parts of PySide6 for #929 - UML documentation	4 h	5 h	6 h	5 h	3 h 30 min
4. Config and setup: <code>ruff</code> <code>mypy</code> <code>pytest</code> <code>pre-commit</code> hook	1 h	1 h	1 h	1 h	30 min
5. Analyzing code/output: - Determine file types for drag-n-drop - Reviewing PRs - Study mocking in <code>pytest</code>	4 h 30 min	2 h	2 h	2 h	4 h 35 min
6. Writing tests	0 h	9 h	3 h	8 h	0 h
7. Writing code: - Implement backend for drag-n-drop web files - Implement persistence for window size, sidebar width, and sorting options - Creating pseudocode	5 h	0 h	4 h	0 h	5 h
8. Running code: Trying out the software	1 h	0 h	1 h	30 min	1 h
Total time spent	20 h 15 min	21 h	21 h	20 h 30 min	20 h 50 min

4 Overview of Issues and Work Done

Within the time frame for this assignment, we managed to work on two issues.

4.1 First Issue

NOTE: This issue was created twice and marked as a duplicate.

Titles and URLs:

- [\[Bug\]: Dragging files from web browser #572"](#)
- [\[Feature Request\]: Grab and download images from web #1288](#)

Overview:

Currently, only local files can be imported by dragging and dropping them onto the application window. The linked issues request that the program should allow for drag-n-drop with files from the browser.

Before our fix, the process after dragging and dropping a URL was:

1. Loop through the URLs.
2. If a given URL is not a local file (i.e., any other type): ignore it.
3. Append the file path to a list.

After collecting all the file paths:

4. If a file name already exists in the library: ask the user if they want to rename it, skip it, or replace it.
5. Begin copying the files to the library.

Out fix:

Our fix is that, instead of skipping the non-local URLs, we check if it is an HTTP or HTTPS URL. In that case, we download it to a temporary directory and append its path to the list of local files. Because the file is in fact downloaded, we treat it as any other local file. Finally, after copying all files, we delete the downloaded file and its temporary directory (because by then, it has already been copied, and it was never meant to be saved somewhere else on disk).

The reason we use temporary directories for all the dropped web URLs is that multiple files may have the same name (such as default.jpg, thumbnail.png). If these were placed in the same directory, we would have to resolve the duplicate naming issues ourselves. But, by placing them in separate directories, instead reuse the duplicate file name handling that is already implemented in the program.

4.2 Second Issue

Title and URL:

- [\[Feature Request\]: Preserve changed window sizes and sorting options on relaunch](#)

Overview:

At the moment, after quitting and reopening TagStudio the window size, sidebar width, and sorting options revert to their defaults. The requested feature is that the previous window, sidebar width sorting options should be restored from the last usage.

Our fix:

Upon shutdown, the values for the window size, sidebar width and sorting options are saved to a file which, upon reopening the program, can be read and used to initialize these attributes to the same values.

5 Requirements for the New Features

Issue	ID	Title	Description	Test Case
572	R1	Only handle files	The system shall only accept files when dragged from a browser, ignoring other types of URLs or non-file content.	<code>test_save_web_file_non_downloadable_content_disposition_returns_none</code> : Checks that if the content disposition does not contain attachment/inline, it should not be downloadable because it is not a valid web file and therefore return none and not create a temp directory.
572	R2	Download to temporary folder	Files dragged from the browser shall first be downloaded to a temporary directory before being added to the TagStudio library.	<code>test_save_web_file_attachment_downloads_to_temp_directory</code> : if content disposition contains correct content (attachment/inline) meaning a valid attachment, the function <code>save_web_file</code> should write the file into a temp directory and return its path.
572	R3	Add file to TagStudio library	After downloading, the system shall add the file to the TagStudio root library so it is visible in the gallery.	<code>test_web_file_added_to_library</code> : test if the file downloaded is in the library
572	R4	Delete temporary folder	After importing, the temporary folder is deleted.	<code>test_temp_folder_deleted_after_import</code> : test if the temporary directory is deleted
572	R6	Failure Handling	If the URL is invalid or the download fails, the system should return None.	<code>test_save_web_file_returns_none_on_failed_download</code> : Tests that <code>DropImportModal.save_web_file()</code> returns None when the download fails (e.g., when <code>response.ok</code> is False).
929	R7	Save window size persistently	The window size should be stored when the application is closed.	<code>test_window_size_is_saved_on_close</code> : Test if when we close a window and then open a new window, the user settings (height and width) are saved
929	R8	Restore window size	The system should restore the saved window dimensions upon startup.	<code>test_window_size_is_saved_on_close</code> : Test if when we close a window and then open a new window, the user settings (height and width) are saved
929	R9	Save sidebar size persistently	The sidebar size should be stored when the application is closed.	NOTE: Not finished due to time constraints.
929	R10	Restore sidebar size	The system should restore the saved sidebar dimensions upon startup.	NOTE: Not finished due to time constraints.
929	R11	Save sorting options persistently	The sorting options should be stored when the application is closed.	<code>test_cache_ui_state_stores_sorting_options</code> checks that <code>QtDriver.cache_ui_state()</code> writes

				the current sorting mode and sorting direction to cached settings when closing the app
929	R12	Restore sorting options	The system should restore the saved sorting options upon startup.	<code>test_restore_sorting_options_from_cache</code> checks that <code>QtDriver.init_library_window()</code> restores valid cached sorting mode and direction.
929	R13	Use default settings	If no previously stored values exist for the window size, sidebar size, or sorting options, the system should use the default configuration for each respective setting.	<code>test_no_cache_uses_defaults</code> checks that <code>QtDriver.init_library_window()</code> falls back to default sorting and thumbnail behavior and does not restore cached sizes when no cache values are to be found.
929	R14	Handle invalid settings	If the saved values for window size, sidebar size, or sorting options are invalid, the system should use the default configuration for each respective setting.	<code>test_invalid_cache_uses_defaults</code> checks that <code>QtDriver.init_library_window()</code> ignores invalid cached values and uses safe defaults instead.

6 Code changes

GitHub comparisons between our patches and the original repo's main branch:

- [Patch for issue #572](#)
- [Patch for issue #929](#)

The patches pass all automated checks. This can be verified by running:

- `ruff check`
- `ruff format`
- `mypy --config-file pyproject.toml`
- `pytest tests/`

7 Architecture of the Program

TagStudio uses a root directory for file and tag management, called a library. This directory contains a “TagStudio” subdirectory that holds cached preview thumbnails of files and a SQLite database for storing tags and file entries. This allows for creating separate tags for different libraries.

Importing a file copies it to the root directory, which means that adding tags to it does not modify the real file, and does not add extra metadata files alongside the real file.

The architecture of TagStudio can be represented as four components: presentation layer, application layer, storage layer, and file system.

1. Presentation Layer

- Built with the Qt framework.
- This layer generates input for the application layer. For example, inputs for:
 - Importing files
 - Browsing files
 - Creating and modifying tags
 - Searching and filtering
 - Displaying metadata

2. Application Layer

- Receives input from the presentation layer.
- Examples of responsibilities:
 - Execute imports
 - Apply modifications to tags
 - Evaluate searching and modifying
 - Caching files and tags
 - This layer is independent of the UI logic from the presentation layer. This way, it is easier to maintain, debug, and test.

3. Storage Layer

- All tags and file info are stored in an SQLite database.
- Examples of data stored:
 - Entries (file path references, not binary data)
 - Tag definitions, colors, aliases
 - Relationships between parent-child tag, and tag-file
 - This layer does not modify the files, but references the files in the next layer.

4. File System

- Finally, the storage layer accesses the user’s file system to:
 - Import files
 - Generate thumbnail previews

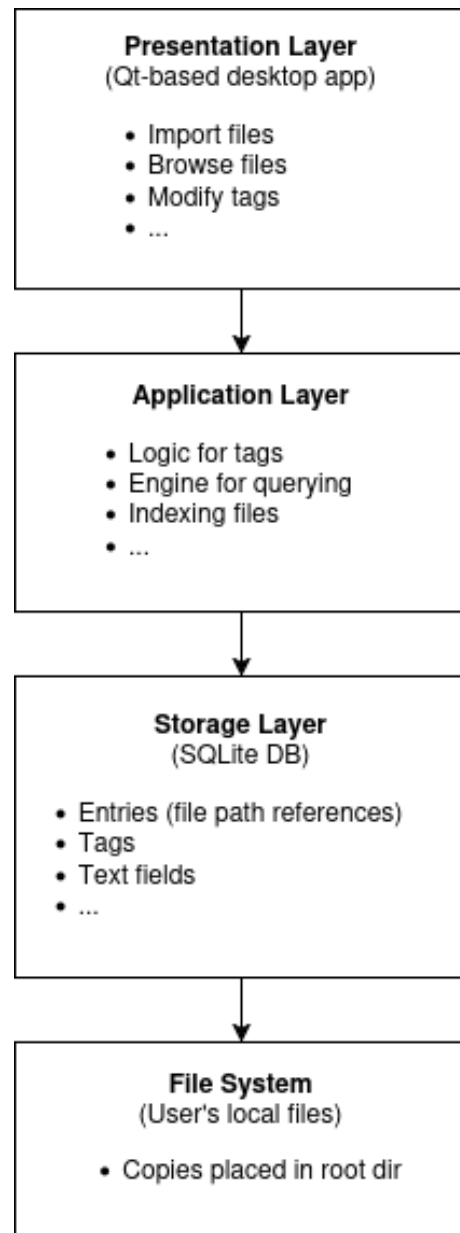


Figure 1: Architecture of TagStudio.

8 UML

8.1 UML Class Diagram for Issue #572

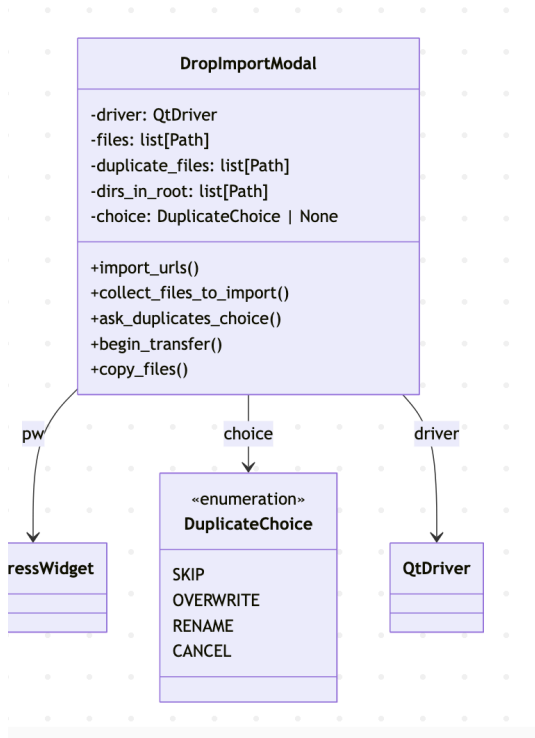


Figure 2: DropImportModal before fixing.

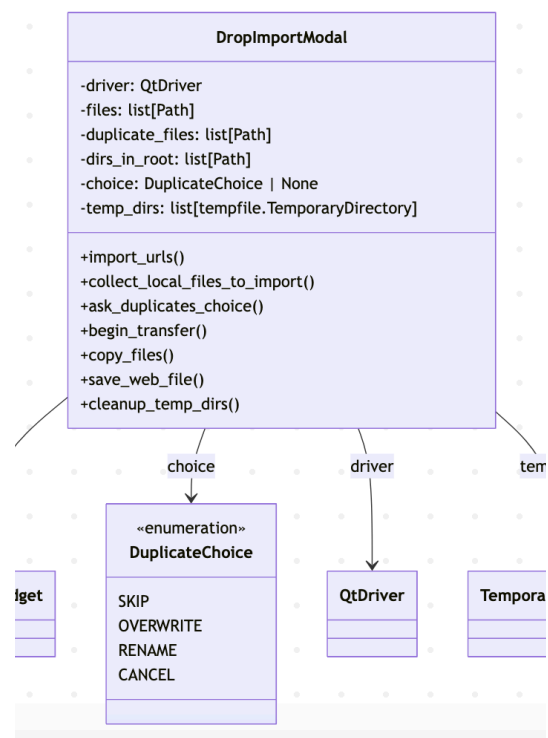


Figure 3: DropImportModal after fixing.

Figure 2 and Figure 3 illustrate the structure of the `DropImportModal` class before and after the changes that handled issue #572 were made. It highlights the key methods, attributes, and dependencies relevant to the import of files and the handling of duplicates.

`DropImportModal` is the central class responsible for importing files into the created library. It originally maintained a state for the files being imported, tracked duplicates, and interacted with other components to manage file transfer and other user choices.

Regarding `DropImportModal`'s attributes, the main and relevant ones (which are the ones documented in the diagram) are:

- `driver`:
 - a `QtDriver` object (which manages application-level operations).
- `files`, `duplicate_files`, `dirs_in_root`:
 - lists that store the files and directories being imported.
- `choice`:
 - stores the user's selection for handling duplicate files.
- `temp_dirs` (only in Figure 3):
 - stores temporary directories for downloaded web files.

The diagram includes the core public methods:

- `import_urls()`, `collect_files_to_import()`, `ask_duplicates_choice()`, `begin_transfer()`, `copy_files()`:
 - These handle file collection, duplicate handling, and transferring files to the library.

- `save_web_file()`, `cleanup_temp_dirs()` (in Figure 3):
 - These handle downloading files from web URLs and cleaning up temporary storage.

8.2 UML Class Diagram for Issue #929

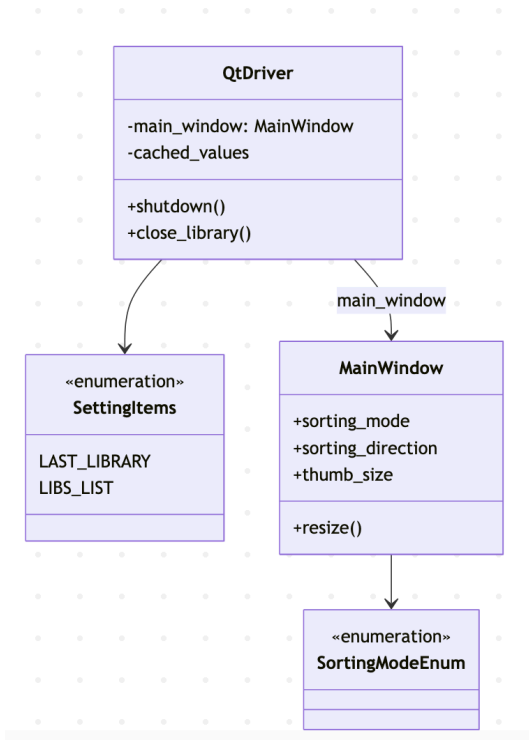


Figure 4: DropImportModal before fixing.

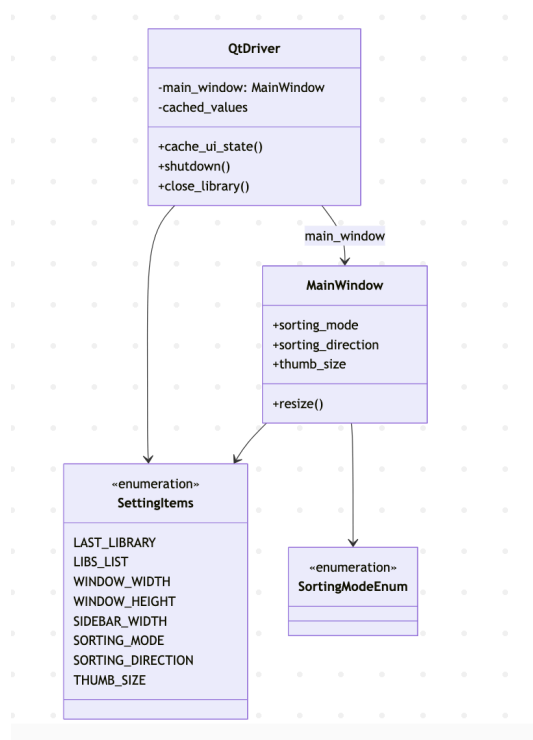


Figure 5: DropImportModal after fixing.

Figure 4 shows that the UI-related values (like window size, etc.) were not persisted, nor did the `QtDriver` class store the UI state.

`SettingsItems` did not contain keys for window settings, etc. The UML diagram on the right shows the architectural structure after the changes have been implemented that supports a persistent UI state. Enumeration values have been added to `SettingItems` to represent what the issue specified should be persisted (window dimensions, sidebar width, etc.), and new methods like `cache_ui_state()` have been introduced to `QtDriver` to save the current UI state.

Regarding attributes, the main and relevant ones (which are the ones documented in the diagram) are:

- `main_window`:
 - A reference to the `MainWindow` instance (the primary graphical user interface).
- `cached_values`:
 - It is used to save and retrieve UI related values between sessions.
- `sorting_mode`:
 - Indicates if the items were sorted in ascending or descending order.
- `sorting_direction`:
 - Indicates if the items were sorted in ascending or descending order.
- `thumb_size`:

- Represents the current thumbnail size.

The diagram includes the core public methods:

- `shutdown()`:
 - Handles application shutdown.
- `close_library()`:
 - Handles the closing of the currently open library.
- `cache_ui_state()` (only in Figure 5):
 - Saves the current window dimensions, sidebar width, sorting settings, and thumbnail size to persistent storage.
- `resize()`:
 - Adjusts the window dimensions.

9 Overall Experience

One of the main experiences we take with us from this project is how resolving an issue in a real open-source project can be a lot more complex than it initially might appear. Even though the change itself might be quite small in terms of lines of code, understanding the surrounding architecture, identifying requirements, and ensuring that existing behavior is preserved can require substantial effort. We also gained experience in working with regression tests, tracing requirements to actual code and tests, as well as collaborating effectively and in a structured way.

For this final assignment, we would assess both our Team and Way of Working alphas as being closer to the *Performing* and *Working Well* states, respectively, than in previous assignments. Compared to assignments 1 and 3, where our Way of Working was mostly in the *In Place* state, we feel like our process has now become more natural and structured. We now work efficiently using the agreed-on practices (e.g., branching, commit structure) and feel like they've become well integrated into how we work. For this last assignment, it feels more like we collectively own our solutions since we had more time to discuss implementations together this time, compared to earlier assignments.

Regarding the Team alpha, we believe that we have now reached the *Performing* state. Looking back at all assignments, we have demonstrated that we can adapt to changing contexts and requirements while consistently delivering work we are proud of. We have also become better at minimizing wasted work by clarifying requirements early, coordinating tasks more effectively, and reviewing each other's contributions continuously. Compared to Assignment 2, where we identified ourselves as *Collaborating*, we now function as a cohesive and self-managing unit.

9.1 Our Work in Context with Best Software Engineering Practices

The work that was carried out for solving the two issues #572 and #929 resulted in multiple benefits when analyzing using the SEMAT kernel alphas. Specifically **requirements**, **software systems**, **work**, **stakeholders**, and **opportunity**. The opportunity perspective could be reasoned from the standpoint of, why is this work worth doing? In this case, the opportunity was to improve usability in TagStudio by fixing a commonly reported drag-and-drop limitation and ensuring consistent interface behavior across sessions.

One benefit was that the implemented changes and features improved the usability of the system/program in a way that the system/program became more versatile (more valuable features). For instance, the drag-and-drop web file handling (issue #572) introduces clearer requirement boundaries by using the HTTP content disposition header to distinguish downloadable files from unsupported web content and also having safe failure handling. This, in turn, strengthens the requirements alpha by making the expected system behaviors more explicit and testable.

The persistent UI state implementation (issue #929) improves the stakeholder experience by maintaining user preferences across sessions (storing window sizes, sidebar dimensions, etc), going more towards real software that is closer to user expectations and workflow. Interactions with the project maintainers through the GitHub issue tracker also contributed to the stakeholders and requirements alpha positively. This communication gave access and project expectations and reduced the risk of implementing an unwanted solution.

Regarding the software system perspective, the automated regression tests increased coverage and confidence. Separating responsibilities between URL handling, temporary file storage, and local file collection improves maintainability and reduces dependencies on components. The refactoring (in this case, the addition of new features, implementation, and tests) did not cause any unwanted or unintended side effects. The separation between, for instance, the downloading, importing, and cleanup logic improves maintainability and gives room for further development in an easy manner.

But there are still some drawbacks/negatives and limitations. Relying on the content disposition header assumes that there is consistent server behavior and could reject otherwise downloadable resources. Regarding work, alpha onboarding will require an immense amount of effort to understand the Qt architecture, all the tests, frameworks, and contribution workflow. But in the end, the work shows typical trade-offs between usability and complexity in software engineering practices.